claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-aff180fc66cdb7b9f.jsonl`
- SHA-256: `2ed61e6b2dda8e2a71466f28cdad81d62e81e47eedf63b3b4bade6b4eb9cf70c`
- Source modified: `2026-06-21T13:38:42+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:34:48.455Z)

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app). khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

AREA: Config system + deployment profiles + the decoupled-compute seam. READ: backend/config/ (models.py, presets.py, startup.py, the loader + precedence), docs/COMPUTE_DECOUPLED_SERVING/README.md, docs/archives/decisions/DECISIONS.md (ADR-012/013/014). Answer: how deployment.profile + compute_target select truly-local vs cloud-serving vs cpu-mock; the config precedence (defaults<preset<config.json<env); what knobs a packaged app must expose to a NON-TECH user vs hide; how a single build can run BOTH local and cloud by config alone.

2. assistant (2026-06-21T13:34:52.806Z)

I'll analyze the config system, deployment profiles, and the decoupled-compute seam. Let me start by reading the key files.

3. user (2026-06-21T13:34:55.590Z)

fatal: Cannot fast-forward to multiple branches.

4. user (2026-06-21T13:34:55.614Z)

```
1       """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3       A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4       config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5       knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7       Precedence (lowest ? highest), realised by the loader::
8
9           built-in model defaults  <  profile preset  <  config.json  <  env  <  worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
```

```
    # that EXIST today ? M1 is pure/additive: the new ``deployment.{profile,compute_target}`` plus
    # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones extend
    # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
    # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
    PROFILE_PRESETS: dict[str, dict[str, Any]] = {
        "truly-local": {
            "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
            # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU box).
            # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
            "indexing": {"skip_ai_handoff": True},
            "storage": {"backend": "local"},
        },
        "cloud-serving": {
            "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
            # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor (D11).
            "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
            "storage": {"backend": "gcs"},
        },
        "cpu-mock": {
            "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
            # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression profile.
            "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
            "storage": {"backend": "local"},
        },
    }


    def is_known_profile(profile: str) -> bool:
        """True for a non-empty, recognised profile name."""
        return bool(profile) and profile in PROFILE_PRESETS


    def preset_for(profile: str) -> dict[str, Any]:
        """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate the
        module-level table), or ``{}`` for ``""`` / an unknown profile."""
        return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))


    def _truthy(val: Optional[str]) -> bool:
        """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
        if val is None:
            return False
        return val.strip().lower() in {"1", "true", "yes", "on"}


    def _cuda_available() -> bool:
        """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here and
        ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no GPU'."""
        try:
            # Intentionally lazy + local: keeps the config-load path torch-free (torch is heavy
            # and optional). Only reached when a caller opts into probe_cuda.
            import torch

            return bool(torch.cuda.is_available())
        except Exception:
            return False
```

```
95
96
97      def probe_cuda_available() -> bool:
98          """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101         that *wants* the GPU heads-up can pass the result in without the startup module
importing
102         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105         error that could false-fail."""
106         return _cuda_available()
107
108
109     def suggest_profile(
110         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111     ) -> str:
112         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114         Signals, in priority order:
115
116           - ``K_SERVICE`` set (Cloud Run)        ? ``"cloud-serving"``
117           - ``CI`` truthy                        ? ``"cpu-mock"``
118           - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119           - otherwise                            ? ``""`` (no strong suggestion)
120
121         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124         """
125         e = os.environ if env is None else env
126         if e.get("K_SERVICE"):
127             return "cloud-serving"
128         if _truthy(e.get("CI")):
129             return "cpu-mock"
130         if probe_cuda and _cuda_available():
131             return "truly-local"
132         return ""
133
```

5. user (2026-06-21T13:34:55.614Z)

```
1       """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3       A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a
coherent
4       runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
5       (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7         * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one
config bug we
8           can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9           bucket can never serve). This is the README §4.3 "loud error before any job".
10        * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
11          falling back to CPU) ? a **warning**, never an error: the probe is best-effort
```

```
(torch may live
         only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
         own healthcheck is the real authority. A warning surfaces it early without false-
failing.
       * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on
Cloud Run) ?
         a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
         we can't probe offline, so the storage backend's own init is the authority; we just
flag it.

     **DEFAULT-OFF:** with no profile selected (``profile == ""``) this is a strict **no-op**
? exactly
     the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors
the loader,
     which applies a preset only for an explicit profile). So nothing changes for today's
manual-knob
     deployments.
     """

     from __future__ import annotations

     import logging
     import os
     from dataclasses import dataclass
     from typing import Any, Mapping, Optional

     LEVEL_ERROR = "error"
     LEVEL_WARN = "warn"

     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
     _CLOUD_STORAGE = ("gcs", "s3")
     # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /
     # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
config dir
     # with no active login), so they would suppress the heads-up falsely. (s3 creds are
intentionally
     # left to boto3's default chain ? no parallel heads-up, by design.)
     _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")


     @dataclass(frozen=True)
     class StartupCheck:
         """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
up)."""

         level: str
         code: str       # stable, greppable code (e.g. "STORAGE_INCOHERENT")
         message: str


     class StartupCheckError(RuntimeError):
         """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL
check."""

         def __init__(self, failures: list[StartupCheck]):
             self.failures = failures
             super().__init__(
                 "deployment profile startup check failed: "
                 + "; ".join(f"[{c.code}] {c.message}" for c in failures)
             )


```

```
    def _creds_discoverable(env: Mapping[str, str]) -> bool:
        """True if *some* Google ADC source is hinted by the environment. Best-effort ? a
False here
        only downgrades to a WARNING, never a hard failure (the storage init is the
authority)."""
        return any(env.get(k) for k in _GCP_ENV_HINTS)


    def run_startup_checks(
        config: Any,
        *,
        cuda_available: Optional[bool] = None,
        env: Optional[Mapping[str, str]] = None,
    ) -> list[StartupCheck]:
        """Return the per-profile findings for ``config`` (empty when no profile is
selected).

        ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ?
GPU checks
        are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available()``).
        Pure: no IO, no torch import ? the caller owns the probe so this stays importable
everywhere.
        """
        profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
        if not profile:
            return []  # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).

        e = os.environ if env is None else env
        storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
        indexing = getattr(config, "indexing", None)
        detector_impl = getattr(indexing, "detector_impl", "auto")
        device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

        checks: list[StartupCheck] = []

        if profile == "truly-local":
            # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud
backend here is
            # an inconsistent (but not fatal) choice worth surfacing.
            if storage_backend not in ("local", "gdrive"):
                checks.append(StartupCheck(
                    LEVEL_WARN, "STORAGE_UNUSUAL",
                    f"profile 'truly-local' usually uses local/gdrive storage, but
storage.backend="
                    f"'{storage_backend}'. Reads will go to a remote backend.",
                ))
            # GPU heads-up (warning only ? the detector healthcheck is the authority; the
probe is
            # best-effort and may be False even when a subprocess detector env has CUDA).
            if cuda_available is False:
                if device == "cuda":
                    checks.append(StartupCheck(
                        LEVEL_WARN, "CUDA_NOT_VISIBLE",
                        "device='cuda' but no CUDA GPU is visible to this process. If the
box truly "
                        "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
                        "'auto' for the CPU fallback (M4).",
                    ))
                elif device == "auto":
                    checks.append(StartupCheck(
                        LEVEL_WARN, "CPU_FALLBACK",
                        "device='auto' and no CUDA GPU is visible to THIS process ? auto may
resolve "
```

```
                        "to CPU (much slower). If torch lives only in the detector
subprocess env this "
                        "can be a false alarm; the runner's DeviceContext/healthcheck is the
authority.",
                    ))

        elif profile == "cloud-serving":
            # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't
serve.
            if storage_backend not in _CLOUD_STORAGE:
                checks.append(StartupCheck(
                    LEVEL_ERROR, "STORAGE_INCOHERENT",
                    f"profile 'cloud-serving' requires a cloud storage backend "
                    f"({'/'.join(_CLOUD_STORAGE)}), but storage.backend='{storage_backend}'.
"
                    "A cloud job cannot read/write a local-only backend.",
                ))
            elif storage_backend == "gcs" and not _creds_discoverable(e):
                checks.append(StartupCheck(
                    LEVEL_WARN, "CREDS_UNCONFIRMED",
                    "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
                    "Default Credentials. The GCS backend will fail loudly at init if ADC
can't resolve.",
                ))
            if device == "cpu":
                checks.append(StartupCheck(
                    LEVEL_WARN, "CLOUD_CPU_DEVICE",
                    "profile 'cloud-serving' runs on a GPU (L4) but
indexing.wasb.device='cpu' ? "
                    "detection will not use the GPU.",
                ))
            elif cuda_available is False and device in ("cuda", "auto"):
                checks.append(StartupCheck(
                    LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
                    "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to
this process.",
                ))

        elif profile == "cpu-mock":
            # The mock profile is for CI/regression ? a real detector here defeats the
purpose.
            if detector_impl != "stub":
                checks.append(StartupCheck(
                    LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
                    f"profile 'cpu-mock' normally uses detector_impl='stub', but
detector_impl="
                    f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
                ))

        return checks


    def enforce_startup_checks(
        config: Any,
        *,
        cuda_available: Optional[bool] = None,
        env: Optional[Mapping[str, str]] = None,
        raise_on_error: bool = True,
        logger: Optional[logging.Logger] = None,
    ) -> list[StartupCheck]:
        """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
        ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.

        Returns the full findings list (so a caller can inspect/surface them). A no-op with
```

```
no logging
171         and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
172         log = logger or logging.getLogger(__name__)
173         checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
174         for c in checks:
175             if c.level == LEVEL_ERROR:
176                 log.error("[STARTUP] %s: %s", c.code, c.message)
177             else:
178                 log.warning("[STARTUP] %s: %s", c.code, c.message)
179         errors = [c for c in checks if c.level == LEVEL_ERROR]
180         if errors and raise_on_error:
181             raise StartupCheckError(errors)
182         return checks
183
```

6. user (2026-06-21T13:34:56.049Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
```

```
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
73          # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74          device: str = "cuda"        # auto | cuda | mps | cpu
75          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
76          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79          keep_frames: bool = False
80          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81          min_free_gb: float = 0.0
82          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87          # Tri-state:
88          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91          stream_video: Optional[bool] = None
92
93
94      class FusionConfig(BaseModel):
95          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
```

```
97          Every default reproduces control behaviour: the fusion segmenter persists the
98          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99          true ? the person-cue features, but it does NOT gate the served handoff unless a
100         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102         supplied ? off-court persons (spectators / adjacent courts) are excluded.
103         """
104         # Which trajectory substrate seeds the windows (shuttle stays primary).
105         substrate: Literal["wasb", "tracknet"] = "wasb"
106         # Windowing velocity threshold for the fusion family (the engine reads
107         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109         velocity_threshold: float = 0.02
110         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111         # enabled ? so the default path never imports torch / touches the GPU.
112         cue_flags: dict[str, bool] = Field(default_factory=dict)
113         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115         min_crossings: int = Field(default=0, ge=0)
116         # Served Gate B: when the person cue is on, drop windows with median in-court
players
117         # < this. 0 = OFF.
118         min_players: int = Field(default=0, ge=0)
119         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120         court_polygon: Optional[list[list[float]]] = None
121         # Net line for the person two-sided-motion split (person features only ? the shuttle
122         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123         net_axis: Literal["x", "y"] = "y"
124         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0
141         detector_model: str = "fasterrcnn_resnet50_fpn"
142         detector_score_threshold: float = 0.5
143         yolo_velocity_threshold: float = 0.15
144         yolo_frame_skip: int = 3
145         yolo_tracker: str = "bytetrack.yaml"
146         yolo_prefetch_workers: int = 2
147         min_action_window_duration: float = 2.0
148         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151         # several rallies + the dead time between them. Never merges, only cuts (recall
```

```
can't drop).
152         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155         max_window_duration: float = 6.0
156         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
157         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160         # Only cuts (recall can't drop). OFF by default until measured/promoted.
161         window_hard_cap: bool = False
162         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169         window_inactivity_end: float = 1.5
170         inactivity_rally_thresh: float = 0.20
171         inactivity_min_density: float = 0.30
172         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
```

```
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
        # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
        # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
        # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
        # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
        skip_ai_handoff: bool = False
        # Optional debug knob: trim every video to the first N seconds before processing.
        debug_duration: Optional[float] = None

        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
```

```
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True           # faint backing box behind the text for legibility
251         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
252         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
267         min_rally_duration: float = Field(default=0.0, ge=0.0)
268         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
```

```
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = ""        # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = ""         # the CF Access application AUD tag the token must
carry
306         # CORS allow-origins. Default ["*"] keeps the local web UI working; tighten to the
hosted
307         # origin(s) in multi-tenant mode. (credentials stay OFF ? Cf-Access is a header, not
a cookie.)
308         cors_allow_origins: List[str] = ["*"]
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
```

```
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375         inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
379         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
380
381         enabled: bool = False
382         pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to
a real one
383         fx_inr_per_usd: float = 83.0                  # USD?INR display rate (Bangalore
market)
384         margin: float = 0.0                            # resale markup (?0); 0 = bill at
cost
385
386
387     class AppConfig(BaseModel):
388         """Root configuration object.
389
390         All runtime configuration should be accessed via an instance of this class
391         (or the loader) rather than raw dict lookups.
392         """
393
```

```
        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)
        storage: StorageConfig = Field(default_factory=StorageConfig)
        deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
        billing: BillingConfig = Field(default_factory=BillingConfig)

        # Allow extra keys during transition so existing config.json files do not break.
        model_config = {
            "extra": "allow",
            "validate_assignment": True,
        }

        # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
        # attribute access / the typed config. A nested *section* is returned as a plain
dict
        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
        # section model instead breaks it (a model has no .get), which bit every validator
on
        # real runs. Scalar leaves are returned as-is.
        def get(self, key: str, default: Any = None) -> Any:
            _MISSING = object()
            val: Any = _MISSING
            if key in type(self).model_fields or hasattr(self, key):
                val = getattr(self, key, _MISSING)
            if val is _MISSING:
                # Legacy convenience: find a leaf key inside any nested section. Sections
are
                # discovered dynamically from the model fields, so a future top-level
section
                # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val

        def __getitem__(self, key: str) -> Any:
            val = self.get(key)
            if val is None and key not in type(self).model_fields:
                # Re-raise like a real dict for unknown top-level
                raise KeyError(key)
            return val


    # Convenience type alias for callers that want to annotate
```

```
451     Config = AppConfig
452
```

7. assistant (2026-06-21T13:34:59.905Z)

```
Let me read the loader and the decoupled-compute README and ADRs.
```

8. user (2026-06-21T13:35:04.629Z)

```
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py
```

9. user (2026-06-21T13:35:04.947Z)

```
backend\config\loader.py
backend\config\presets.py
backend\config\__init__.py
backend\config\startup.py
backend\config\models.py
```

10. user (2026-06-21T13:35:05.137Z)

```
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
docs\COMPUTE_DECOUPLED_SERVING\runlogs\README.md
docs\COMPUTE_DECOUPLED_SERVING\architecture.svg
docs\COMPUTE_DECOUPLED_SERVING\runlogs\RUNLOG_truly-local_TEMPLATE.md
docs\COMPUTE_DECOUPLED_SERVING\README.md
```

11. user (2026-06-21T13:35:09.494Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      import os
14      from pathlib import Path
15      from typing import Any
16
17      from pydantic import BaseModel
18
19      from .models import AppConfig
20      from .presets import (
21          COMPUTE_TARGET_FOR_PROFILE,
22          PROFILE_PRESETS,
23          is_known_profile,
24          preset_for,
25          suggest_profile,
26      )
27
28      logger = logging.getLogger(__name__)
29
30      DEFAULT_CONFIG_PATH = Path("config.json")
```

```
31
32
33      def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
34          """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
35          replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
36          overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
37          path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
38          out = dict(base)
39          for key, val in override.items():
40              if isinstance(val, dict) and isinstance(out.get(key), dict):
41                  out[key] = _deep_merge(out[key], val)
42              else:
43                  out[key] = val
44          return out
45
46
47      def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
48          """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
49          is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
50          environment never silently selects a profile (and never silently spends on cloud).
51
52          An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
53          profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
54          asserting a preset that was never applied. A bad value in config.json is left to
surface as
55          a hard, typed-Literal error downstream (config-file correctness)."""
56          env_profile = os.environ.get("DEPLOYMENT_PROFILE")
57          if env_profile and env_profile.strip():
58              ep = env_profile.strip()
59              if is_known_profile(ep):
60                  return ep
61              logger.warning(
62                  "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
63                  "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
64              )
65              # fall through to the file's profile (env typo must not suppress a valid file
choice)
66          dep = file_data.get("deployment")
67          if isinstance(dep, dict):
68              prof = dep.get("profile")
69              if isinstance(prof, str) and prof.strip():
70                  return prof.strip()
71          return ""
72
73
74      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
75          """Dotted paths in `data` that the typed config will not honor.
76
77          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
78          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
79          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
80          user's intent is lost ? collect every such key so load_config can warn.
81          """
82          unknown: list[str] = []
83          fields = model_cls.model_fields
84          for key, val in data.items():
```

```
            if key not in fields:
                unknown.append(prefix + key)
                continue
            ann = fields[key].annotation
            if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
                unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
        return unknown


    def _get_builtin_defaults() -> dict[str, Any]:
        """Return a plain dict of the current built-in defaults.

        The AppConfig model is the single source of truth for defaults.
        """
        return AppConfig().model_dump(mode="json")


    def load_config(path: str | Path | None = None) -> AppConfig:
        """Load configuration with the following precedence:

        1. Built-in defaults defined in the Pydantic models.
        2. Values from the JSON file (if present and readable).
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
        # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
        # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
        # loader's "bad input is non-fatal" contract.
        if not isinstance(file_data, dict):
            logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
                           cfg_path, type(file_data).__name__)
            file_data = {}

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
```

```
at
144         # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
145         # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
146         # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
147         defaults = _get_builtin_defaults()
148         profile = _resolve_explicit_profile(file_data)
149
150         if profile and not is_known_profile(profile):
151             # Only an unrecognised value from config.json reaches here (env typos already
fell back
152             # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
153             # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
154             logger.warning(
155                 "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
156                 profile, ", ".join(PROFILE_PRESETS),
157             )
158             profile = ""
159
160         if profile:
161             merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
162             # Record the profile actually applied ? the env may have chosen it over
config.json,
163             # so re-stamp it onto the merged deployment section to keep the record honest.
164             merged.setdefault("deployment", {})
165             if isinstance(merged["deployment"], dict):
166                 merged["deployment"]["profile"] = profile
167                 ct = merged["deployment"].get("compute_target")
168                 canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
169                 if not ct and canonical:
170                     # An active profile with no explicit compute_target (e.g. config.json
blanked it
171                     # to "") gets its canonical target back ? the record never silently lies
about an
172                     # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
173                     merged["deployment"]["compute_target"] = canonical
174                 elif ct and canonical and ct != canonical:
175                     logger.warning(
176                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
177                         "config.json overrides it to %r ? honoring the explicit override.",
178                         profile, canonical, ct,
179                     )
180                 logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
181                             profile, merged["deployment"].get("compute_target"))
182         else:
183             # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
184             merged = {**defaults, **file_data}
185             suggestion = suggest_profile()  # cheap, env-only (no torch); SUGGESTS only
(D15)
186             if suggestion:
187                 logger.debug(
188                     "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
189                     "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
190                     "auto-applies ? D15).", suggestion,
191                 )
```

```

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by scripts/doctor.py diagnostics and the --setup CLI path.
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
        cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
        return cfg_path


    # Backwards-compatible alias used by some older call sites during transition.
    def get_default_config() -> dict[str, Any]:
        """Return defaults as a plain dict (for transitional use only)."""
        return _get_builtin_defaults()

```

12. user (2026-06-21T13:35:10.034Z)

```
"""Configuration package.

Public API:
    from backend.config import load_config, AppConfig, Config

This package is the canonical place for all configuration access.
"""

from .loader import load_config, save_default_config
from .models import (  # re-export common ones
    AppConfig,
    Config,
    DeploymentConfig,
    IndexingConfig,
    StitchingConfig,
    StorageConfig,
    ValidationConfig,
)
from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
from .startup import (
    StartupCheck,
    StartupCheckError,
    enforce_startup_checks,
    run_startup_checks,
)

__all__ = [
    "load_config",
    "save_default_config",
    "AppConfig",
    "Config",
    "DeploymentConfig",
    "IndexingConfig",
    "StitchingConfig",
    "StorageConfig",
    "ValidationConfig",
    "PROFILE_PRESETS",
    "suggest_profile",
    "probe_cuda_available",
    "StartupCheck",
```

```
41          "StartupCheckError",
42          "enforce_startup_checks",
43          "run_startup_checks",
44      ]
45
```

13. assistant (2026-06-21T13:35:12.832Z)

Now let me read the README and the ADRs, plus check where these config fields are actually consumed.

14. user (2026-06-21T13:35:14.160Z)

```
1       # Compute-Decoupled Rally Serving ? one pure core, profile-selected compute + storage
2
3       > **Status:** ? **M0 APPROVED** (owner, 2026-06-21) ? **IN PROGRESS (build M1+)** ·
**Owner:** Avi · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to
`docs/archives/past_projects/` when DONE)
5       > **Tracking anchors:** §7 progress tracker is the source of truth; pointer in memory
`current-state` + `docs/NEXT_STEPS.md` once work starts.
6       > **Relation to existing docs:** the productionization successor to the archived
`DECOUPLED_COMPUTE` (M3/M4); realizes the `ComputeBackend`/`StorageBackend` registries from
[PLATFORM_ARCHITECTURE.md](../PLATFORM_ARCHITECTURE.md) §3; the **truly-local** profile is the
L0 tier of [VIDEO_LOCALITY_MODEL.md](../VIDEO_LOCALITY_MODEL.md) and the `LocalDesktop` backend
of [DESKTOP_APP_PLAN.md](../DESKTOP_APP_PLAN.md).
7       > **Naming:** this was the "Cloud Run + GPU serving subproject" (ADR-012). Renamed
**compute-decoupled serving** because **truly-local is a co-equal, first-class profile**, not an
afterthought.
8       > **Honesty note:** claims tagged `[verified]` (checked against code by the design
workflow `wf_56274ff0`), `[design]` (proposed), `[researched]`.
9
10      ---
11
12      > **? Northstar (D14):** a **mobile, app-like** flow ? point at a **Google Drive** video
? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next
to the source**. Two design promises around it: a user can **switch local?cloud anytime** (D13,
a headline feature), and execution is **never a surprise** ? auto-detect *suggests*, the user
*confirms* (D15).
13
14      ## 0. TL;DR
15      The rally engine is a **pure function of `(input bytes + config) ? outputs`** in three
deterministic stages ? **DETECT ? WINDOW ? STITCH** ? that **no deployment profile may branch
on**. A **`ComputeBackend` (deployment profile)** decides *where* the core runs (a local process
vs a Cloud Run GPU job); a **`StorageBackend`** decides *where bytes come from* (local FS / USB
/ mounted Google Drive / bucket). **One startup config** picks both. We ship **truly-local**
(local GPU + in-process stitch, zero cloud) and **cloud-serving** (upload `<2GB` / gdrive /
bucket ? a Cloud Run GPU job runs detect **and** stitch, so stitch's compute is metered), plus
**cpu-mock** for CI. The single most important caveat: detection is **byte-identical only on the
same GPU** (cross-GPU is sub-pixel-different, per the 2026-06-20 deploy report) ? so the
**parity guarantee is enforced at the WINDOW + STITCH layer**, not detector-CSV bytes.
16
17      ## 1. Problem & goal
18      - **Why now:** DECOUPLED_COMPUTE proved compute *portability* (M0?M2 + M3.5 on a GCP L4)
and is archived; its deferred **M3 (serving endpoint + per-video billing) + M4 (scale-to-zero +
cost guard)** become this project. The owner needs **both** a hosted service *and* a truly-local
mode, with the **core (detect+stitch) unaffected** by which one runs.
19      - **The two user-facing outcomes** (owner, 2026-06-21):
20        - **(a) Hosted service** ? operate on small **local uploads (`<2GB`)** *and* load from
**gdrive / a bucket**, spinning up **Cloud Run for both rally segmentation AND stitching**
(stitching is compute-intensive ? run it where its cost is accounted for).
21        - **(b) Truly-local** ? on a local machine with videos on **local drives/USB + a local
GPU**, run inference **and** stitching **completely locally**.
22      - **"Good" looks like:** the same `video ? rallies ? reel` core gives identical windows
+ stitched ranges whether it runs on a laptop or Cloud Run; switching is a **config/startup
```

choice**, never a code branch in the core.

2. Decisions locked

#	Decision	Source	Implication
D1	The core is a **pure 3-stage function** (DETECT/WINDOW/STITCH); **no profile branch inside it**.	design `[verified]`	All profile difference lives in config + the two registries.
D2	**Two registries:** `ComputeBackend` (where) selected by `deployment.profile` + `compute_target`; `StorageBackend` (what-bytes) by `input_backend`/`storage.backend`.	ADR-014	Realizes PLATFORM_ARCHITECTURE's `local|hosted|byo_worker`.
D3	**Profiles shipped:** `truly-local`, `cloud-serving`, `cpu-mock` (CI); `byo_worker` reserved/deferred.	ADR-014	Enum/seam reserved so BYO isn't precluded.
D4	In **cloud-serving, STITCH runs on Cloud Run** (co-located with detect) so CPU/wall + egress are metered into the per-job cost ledger.	owner (a)	Stitch-on-laptop would hide real cost.
D5	**Parity enforced at WINDOW+STITCH** (byte-exact for a fixed trajectory); **detect** is byte-exact only same-GPU.	deploy report 2026-06-20	Cross-GPU asserted at the rally/window level, never CSV bytes.
D6	**Default-OFF, smallest-safe-first, one PR per milestone**; any core-caller change (e.g. configurable output base) ships behind a flag + a Launch Report.	[[launch-report-convention]]	Zero-regression discipline.
D7	**Cloud Run cold-start: SCALE-TO-ZERO** ? accept the ~1?3 min cold-start (it's an async job: upload?poll?reel, amortized over the multi-minute job). **No warm pool** (that ~$500/mo idle would break per-video billing).	owner 2026-06-21	M6/M7. Revisit a warm lane only on a UX complaint.
D8	**Pricebook = a versioned DB table**; cost computed in **USD** (matches GCP billing = one source of truth), **displayed in INR** at a configurable FX rate (Bangalore market); re-versioned when GCP prices change.	owner 2026-06-21	M8.
D9	**Edge auth = Cloudflare Access** (reuse the site's existing CF). Lock the Cloud Run **ingress to the CF proxy** + **verify the `Cf-Access` JWT signature** (so a direct hit to the Cloud Run URL can't spoof the identity header).	owner 2026-06-21	M9. One identity system.
D10	**Free-tier "data-for-reels" trade:** free reels in exchange for **training rights** (uploaded footage + derived trajectories/labels); **paid tier = no-train** (privacy is the paid feature). **Carve-outs:** train ONLY on attested **ADULT** footage (minor footage ? reel yes, training pool **NO** ? DPDP/GDPR need verifiable parental consent); train on **DERIVED** data + **auto-delete raw**; **ToS counsel-reviewed**; live training-on-uploads **gated to M9** (public signup). Truly-local needs no DPA.	owner 2026-06-21	M9 + D12; ties [[paper-coauthor-aim]] / PERSONALIZATION flywheel.
D11	**Quality floor: Gemini handoff ON** for cloud-serving until the owned model clears the #172 ship-gate (e.g. ?0.70 multi-court) ? then flip via a **Launch Report**; the owned model runs in **shadow/eval** meanwhile. (Gemini = serving/inference ONLY, never a training source ? CLAUDE.md guardrail.)	owner 2026-06-21	M7 + D12.
D12	**Data-flywheel harness is IN SCOPE (owner add, Q5):** capture the (consented, adult) uploaded video + the Gemini reel/rally output ? **reliably store** in the per-video workspace/bucket ? **run shadow evals** (owned-vs-Gemini, dual-eval-set) ? **promote good ones to the golden TRAINING set** (human-reviewed labels) so the owned model climbs toward the D11 floor (then Gemini flips off).	owner 2026-06-21	new **M11**; ties D10 (consent) + `data_pipeline/` + the eval/experiment harness.
D13	**Profile is user-switchable ANYTIME ? a headline FEATURE.** The same user runs **local today, cloud tomorrow, back to local** ? it's just a config/startup choice; the pure core gives **equivalent** output either way. **Advertise it** ("your machine *or* our cloud ? your call, switch anytime").	owner 2026-06-21	Honest caveat: equivalent at the **rally/window** level, not bit-identical across GPUs (D5); a single job runs in one profile, switching is *between* jobs.
D14	**NORTHSTAR ? operate toward this:** a **mobile, app-like** flow ? point at a **Google Drive** video ? **fully cloud-native** run ? the stitched, **ready-to-share** reel lands **back in Drive, next to the source**.	owner 2026-06-21	Implies a **`gdrive-api` (OAuth) StorageBackend** (read source + write reel back) ? distinct from the local **mount** backend; **resolves §8 #7**, new **M12**. The mobile UI is the khelsutra track; the engine must support it (async jobs + status + Drive round-trip / signed URL). Net-new scope (OAuth + Drive write); not necessarily v1, but the design must not preclude it (the `StorageBackend` ABC accommodates it).

42 | D15 | **Auto-detect SUGGESTS, the user CONFIRMS ? execution is NEVER a surprise.** A GPU owner may still choose cloud; **cloud (= spend) is never entered silently**; the active profile is always explicit + surfaced. | owner 2026-06-21 | Amends §4.3 (auto-detect = a proposed default, not an auto-decision). |
43 | D16 | **M6/M7 implementation SHAPE + the cheap §8 picks (owner 2026-06-21):** (a) **Cloud Run JOBS** (not Services-with-GPU) for the detect+stitch unit, **bucket-poll for completion** (reuses `JobWaiting`/`claim_due_job` verbatim, fits async upload?poll?reel + scale-to-zero D7); (b) **stitch ENCODE = libx264** (protects the D5 byte-determinism/parity guarantee + the L4 parity test) ? **NVDEC for DECODE only** (a speed lever that never touches output bytes); **NVENC encode is DEFERRED behind a config knob**, revisit only if M7 cost data justifies relaxing parity (§8 Q8); (c) **input cap = hard-reject `<2GB`** for v1 (a config cap so raising to 4GB later is one line; matches the shipped M2 chunked-upload threshold ? §8 Q9). | owner 2026-06-21 | M6/M7. Resolves §8 Q2 (co-located, = D4), Q8, Q9, Q10 (dir name confirmed). |
44 | D17 | **Real cloud verification BUDGET = $100 / ?10k cap (owner 2026-06-21):** the owner authorized actual GCP spend for due-diligence + production-grade testing/verification of M5?M7 (real L4 run, the M5 runlog, the M7 DEPLOY_REPORT), **provided the cap is respected** ? confirm exact cmd + hourly cost before each spend, prefer the smallest viable run + cheapest-adequate GPU, and **DELETE every resource after** ([[gcp-vm-always-delete]]). Removes the M6/M7 "spend" gate within the envelope; the calibrated pricebook (M8) + counsel ToS (M9) + OAuth app (M12) stay owner/counsel-side. | owner 2026-06-21 | Unblocks the real M5/M7 runs (not just the code) within the cap. |
45
46   ## 3. Foundation ? evolution / ADR lineage ? *trace the idea end-to-end*
47   This project is the latest step in a long decision chain; each ADR contributed a piece and a **subtlety that carries forward**:
48
49 | ADR / doc | Contributed | Subtlety carried into this project |
50 |---|---|---|
51 | **ADR-005** | Permissive licensing (MIT/Apache-2.0/BSD only) | The engine + any served model + the **container image** (ffmpeg, fonts, weights) stay commercial-clean. |
52 | **ADR-008** | Owned-model topology; **train==serve featurizer single-sourcing** | The "core unaffected across profiles" is the same parity discipline, extended from train/serve to **local/cloud**. |
53 | **ADR-009** | KhelSutra Guru; **local-first delivery** | The **truly-local profile** is the direct continuation ? privacy/offline self-host stays first-class. |
54 | **ADR-010** | DECOUPLED: DO?GCP pivot | The cloud-compute origin (GPU + co-located bucket). |
55 | **ADR-011** | DECOUPLED scope = M0?M2+M3.5; **deferred M3 (serving) + M4 (billing/cost-guard)**; overrode the *"rent nothing / not a service"* posture | **This project realizes M3/M4.** Carries §06's owner-gated gates: **DPA/consent for non-owner uploads, collector `md5` keying, WASB-C3**. |
56 | **ADR-012** | GPU-native; TPU deferred; **Cloud Run+GPU scale-to-zero = serving model**; **NVDEC** named as the decode lever | The serving substrate + the 7-step seed scope this expands. |
57 | **ADR-013** | Drop DO; **GCP sole compute stack**; $200 DO credits ? non-compute only | The provider for cloud-serving; DO Spaces remains only a possible S3 *storage* target. |
58 | **? ADR-014 (new)** | **Compute-decoupled serving:** one pure core, profile-selected compute+storage; stitch-where-metered | This doc. **Refines** ADR-011/012, does **not** supersede them. |
59 | PLATFORM_ARCHITECTURE.md | The `ComputeBackend`/`StorageBackend` registries | The abstraction this realizes in code. |
60 | VIDEO_LOCALITY_MODEL.md / DESKTOP_APP_PLAN.md | L0 fully-local tier / `LocalDesktop` backend | The truly-local profile = these, productionized. |
61 | 07-COMPUTE-ABSTRACTION (archived) | `DeviceContext`/`DeviceProfile` (designed) | WASB hardcodes `device='cuda'` with **no CPU fallback** ? a portability gap M4 closes. |
62 | DEPLOY_REPORT_GCP_2026-06-20 (archived) | First real L4 run; **cross-GPU sub-pixel parity finding** | Why D5 (parity at window level, not CSV bytes). |
63
64   ## 4. Design / architecture
65   See [`architecture.svg`](architecture.svg) for the picture. **The core never changes; the box around it does.**
66
67   ### 4.1 The core contract (must stay byte/behaviour-identical) `[verified]`
68   1. **DETECT (GPU):** `DetectorRunner.run_predict(video_path, output_dir, video_id) ? CSV[Frame,Visibility,X,Y]` (`backend/pipeline/detectors/base.py:164`). Three interchangeable

impls ? `WasbRunner` (WSL), `NativeWasbRunner` (Linux GPU), `StubDetectorRunner` (CPU mock) ? emit the **identical CSV schema**. The trajectory CSV is the stable artifact boundary.
69   2. **WINDOW (CPU pure fn):** `trajectory_to_action_windows(points, fps, frame_width, ?) ? windows` (`tracknet_runner.py:281`). A shared `@staticmethod`, zero GPU/IO/randomness ? same trajectory + config = byte-identical candidate windows. Used by every runner *and* the eval/regression stack verbatim.
70   3. **STITCH (CPU/ffmpeg):** `VideoCompiler.compile_highlights(raw_video, segments, out_name) ? reel` (`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? ffprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
71
72   **Non-goal (the trap to avoid):** *no* code branch inside detect/window/stitch keyed on profile ? the same discipline the experiment harness already enforces (a disabled cue is byte-identical to CONTROL).
73
74   ### 4.2 The deployment profiles
75 | Profile | Compute | Storage | Orchestration | When |
76 |---|---|---|---|---|
77 | **truly-local** | `compute_target=local_gpu`; `detector_impl=native` (Linux) / `auto` (WSL); stitch **in-process** | `local` (or `gdrive` = mounted Drive) | `python -m backend.main` or `worker infer` CLI; existing async job worker | Self-host / offline / privacy / dev-on-a-GPU-box ? owner (b) |
78 | **cloud-serving** | `compute_target=cloud_run`; `detector_impl=native` (L4, cuda); **detect AND stitch on the same Cloud Run job** | `gcs` (or gdrive mount / assembled upload); per-video workspace; reels via signed URL | Cloud Run control plane enqueues ? Cloud Run GPU job pulls/runs/pushes; `WAIT_AND_RESUME` reschedules the control-plane job; scale-to-zero | Hosted SaaS ? owner (a) |
79 | **cpu-mock** | `compute_target=cpu_mock`; `detector_impl=stub` (synth/replay) | `local`/in-memory fakes | `run_all_tests.py` / pytest | CI, regression, profile-parity, GPU-free dev |
80 | *byo_worker* (deferred) | tenant GPU via outbound pull agent | tenant's own bucket (signed URLs) | long-poll pull; same job table scoped by tenant | Privacy-sensitive enterprise ? **DEFERRED**, enum/seam reserved |
81
82   ### 4.3 Profile selection (the "startup/setup config") `[design]`
83   ONE new top-level `deployment` section on `AppConfig` + preset application + env auto-detect in `load_config`:
84   - `deployment.profile ? {truly-local | cloud-serving | cpu-mock}` (empty = today's manual knobs, **fully backward-compatible**).
85   - Selecting a profile applies a **coherent preset bundle** of *existing* knobs that today drift independently: **INPUT** (new `input_backend`/`input_root`, parallel to the output-side `StorageConfig`), **COMPUTE** (new `compute_target` enum locking `detector_impl` + `skip_ai_handoff` + workspace strategy), **STORAGE** (`storage.backend`/`workspace_root`/`single_folder_per_video`, all already exist).
86   - **Precedence:** built-in defaults ? profile preset ? `config.json` ? env (`RALLY_DETECTOR_IMPL`, `WASB_*`, `DEPLOYMENT_PROFILE`, ?) ? worker `--set k=v`. Every knob stays individually overridable.
87   - **Env auto-detect SUGGESTS, the user CONFIRMS (D15 ? no surprise execution):** the env gives a *proposed* default (`K_SERVICE` ? cloud-serving; `CI=true` ? cpu-mock; else `torch.cuda.is_available()` ? truly-local), but the user **explicitly confirms or overrides** it ? a GPU owner may still pick cloud, and **cloud (= spend) is NEVER entered silently**. The active profile is always surfaced (logged/shown). **Per-profile startup checks fail/warn fast** (cloud-serving without GCS creds/GPU ? loud error before any job).
88   - The worker (`cmd_infer`/`cmd_train`) is **already profile-agnostic** (reads `config.storage`, accepts `--config`/`--set`) ? no worker rewrite.
89
90   ### 4.4 Compute placement ? and *why stitch runs on Cloud Run* `[design]`
91   DETECT on GPU everywhere; WINDOW CPU-pure everywhere; STITCH is real CPU work. In **cloud-serving, stitch is co-located in the Cloud Run job** so `gpu_active_seconds` (detect) + `wall_seconds` (detect+stitch) + `bytes_out` (reel egress) land on **one metered invocation** ? the per-job cost ledger. Running stitch on a laptop would leave that cost **invisible/unattributed** ? exactly what the owner wants to avoid. (Stitch stays synchronous in the GPU job for the common one-reel case; a queued CPU-only stitch fallback only if compiles get bursty ? the job table + `claim_due_job` already supports both.)
92
93   ### 4.5 Reuse map ? most of the seams already exist `[verified]`

94 **Exists:** the 3-stage core; `_resolve_runner` (the single compute seam); `StubDetectorRunner` (`replay_csv` = byte-exact $0 anchor); `StorageBackend` ABC + 4 backends + `StorageRef.parse_uri`; the worker CLI (fetch/put, `--config`/`--set`); the async job control plane (`jobs` table, `claim_due_job` CAS); `ExecutionPolicy WAIT_AND_RESUME`/`JobWaiting` (carries to Cloud Run dispatch verbatim); the chunked-upload router (`<2GB`); `skip_ai_handoff` (LocalNoAI); storage fakes; golden fixtures + experiment harness. **Partial:** per-video workspace helper (default-OFF); `DeviceContext` (designed); edge-auth (v4 `user_id` columns exist). **Net-new:** the `deployment` section; `input_backend`; configurable output base (kill hardcoded `output/`); Cloud Run dispatch shim + container image; cost ledger + pricebook; edge-auth header extraction; startup env detection.
95
96 ### 4.6 The data-flywheel harness (D12 / M11) `[design]`
97 Gemini-on (D11) ships good reels now; this harness is **how we earn the right to flip Gemini off**. On a **consented, adult** cloud-serving job (the D10 trade), the pipeline additionally:
98 1. **Captures** the source upload + the Gemini reel + the rally output (intervals/report) into the per-video workspace (`workspaces/<video_id>/`), instead of the no-consent auto-delete path.
99 2. **Reliably stores** them durably in the bucket (the consented-retention bucket; raw video kept only for the training pool, not the no-consent path).
100 3. **Runs shadow evals** ? the owned model scores the same video alongside Gemini (`backend/eval/experiment.compare` + the dual-eval-set, [[eval-dual-set-regression]]) ? tracks the owned-vs-Gemini gap per job (the live read on "is the floor reachable yet?").
101 4. **Promotes to golden** ? captured videos that pass review become **golden TRAINING rows** (human-reviewed labels via the annotation flow ? `data_pipeline/`), growing the corpus ? the owned model climbs to the D11 floor ? flip Gemini off via a Launch Report.
102 This is the consent?capture?eval?goldenize **flywheel** (the cloud realization of the PERSONALIZATION contribution model): every consented free reel makes the owned model better. Reuses `data_pipeline/`, `backend/eval/`, and the M9 consent gate; **adults-only**, raw-retention gated by the D10 consent.
103
104 ## 6. Honest limits ? what this does NOT do / prove
105 - A green CI suite proves **plumbing + determinism (window+stitch) + profile-parity** for $0 ? it does **NOT** prove field quality, nor that real-GPU detection is good (that's owner-side, real-video).
106 - **Cross-GPU detection is NOT byte-identical** ? parity is a window-level claim, not a CSV-byte claim.
107 - The cloud `0.1.0-l4` weights are **n=2 plumbing-grade** ? cloud-serving ships with **Gemini handoff ON** until the owned model clears a floor (open Q).
108 - This does not resolve **WASB-C3** (sharing a trained model) or the **DPA/consent** gate ? both remain owner/counsel-gated before public, non-owner serving.
109
110 ## 7. Deliverables & progress tracker ? **source of truth**
111 Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per row**, off `origin/master`, no stacking. Default-OFF where it touches core callers.
112
113 | ID | Deliverable | Depends | Gated? | Status | PR |
114 |----|-------------|---------|--------|--------|----|
115 | M0 | This design doc + project dir + `runlogs/` + ADR-014 | ? | owner sign-off | ? | **? APPROVED 2026-06-21** |
116 | M1 | `deployment.profile` + `compute_target` config + preset bundles + precedence/auto-detect; unit tests (no behaviour change, profile='' default) | M0 | safe | ? | [#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279) |
117 | M2 | `input_backend`/`input_root`; wire main CLI/API input through `StorageRef.parse_uri` (local=identity); storage-fake tests | M1 | safe | ? | [#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280) |
118 | M3 | Configurable output/scratch base ? kill hardcoded `output/` (`trajectory_hybrid:237,313`, `yolo_hybrid:407`, `gemini`); **DEFAULT stays `output/`**; golden + stub-E2E byte-identical | M1 | ? default-OFF + Launch Report on flip | ? | [#282](https://github.com/Khelsutra/badminton-highlight-indexer/pull/282) |
119 | M4 | `DeviceContext` + WASB **CPU-fallback**; unified healthcheck/device wiring | M1 | safe (cuda default unchanged) | ? | [#283](https://github.com/Khelsutra/badminton-highlight-indexer/pull/283) |
120 | M5 | **truly-local profile** end-to-end (preset + startup checks); first committed **runlog** (truly-local sample run) | M2,M3,M4 | owner real-GPU | ? |

[#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286) (preset+startup-checks+L4 parity ?; runlog ? owner GPU) |
121 | M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) + **containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner (GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287) (shim+image+mocked ?; real build/run = M7 ? owner) |
122 | M7 | **cloud-serving profile** e2e on L4: upload`<2GB` + gdrive/bucket ? detect+stitch on Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6 | ? owner (spend) | ? | ? |
123 | M8 | Per-job **cost ledger** (v6 migration: `owner_id, gpu_active_seconds, wall_seconds, bytes_out, cost_usd, cost_breakdown_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288) (ledger+pricebook+/cost ? default-OFF; real prices + usage-emission ? owner/follow-up) |
124 | M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | ? |
125 | M10 | `byo_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |
126 | **M11** | **Data-flywheel harness (D12):** on a consented adult job, **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |
127 | **M12** | **`gdrive-api` (OAuth) StorageBackend (D14 Northstar):** cloud reads the source from + writes the stitched reel **back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |
128
129   **Testing strategy is a first-class deliverable** ? [`TESTING_STRATEGY.md`](TESTING_STRATEGY.md). **Run logs / sample runs for posterity** ? [`runlogs/`](runlogs/).
130
131   ## 8. Open questions ? owner *(blocking-gates vs nice-to-knows)*
132   **? The 5 gating questions are LOCKED (owner, 2026-06-21) ? see §2 D7?D12:**
133   1. ? **Cold-start ? D7** (scale-to-zero, accept cold-start).
134   3. ? **Pricebook ? D8** (versioned DB table; USD internal / INR display).
135   4. ? **Edge auth ? D9** (Cloudflare Access; verify Cf-Access JWT, lock ingress to proxy).
136   5. ? **DPA/consent ? D10** (free-tier data-for-reels trade; adults-only; derived-data; counsel-gated to M9).
137   6. ? **Quality floor ? D11** + the **data-flywheel harness D12 / M11** (Gemini-ON until the owned model clears the floor; capture?store?eval?goldenize meanwhile).
138
139   **? The remaining smaller picks are LOCKED (owner, 2026-06-21) ? see §2 D16:**
140   2. ? **Stitch placement ? co-located** in the detect job (= D4; one metered unit; the job table supports a future CPU-split non-breaking, revisit only on bursty-compile metrics).
141   7. ? **RESOLVED ? D14:** cloud needs a **`gdrive-api` (OAuth) StorageBackend** (read the source from + write the reel back to the user's Drive) ? the Northstar's Drive round-trip ? distinct from the local **mount** backend (which stays for truly-local). New milestone **M12**.
142   8. ? **Codec ? D16:** **libx264 ENCODE** (protect parity) + **NVDEC DECODE-only** (speed, no output-byte change); NVENC-encode deferred behind a config knob.
143   9. ? **Input cap ? D16:** **hard-reject `<2GB`** for v1, as a config cap (raise to 4GB later in one line).
144   10. ? **Directory name ? `COMPUTE_DECOUPLED_SERVING/` CONFIRMED final** (already the name everywhere + in ADR-014); `CLOUD_RUN_GPU_SERVING/` dropped.
145
146   ## 9. Definition of done
147   - [ ] M1?M4 land (config + input + output-base + device layer), each green on golden cross-OS + experiment no-regression.
148   - [ ] **Profile-parity test green** ? truly-local vs cloud-mock produce byte-identical windows + segment ranges.
149   - [ ] **truly-local** (M5) and **cloud-serving** (M7) each have a committed **runlog / DEPLOY_REPORT** with a real sample run.
150   - [ ] Cost ledger (M8) makes GPU + **stitch** + egress cost visible per job; edge auth

(M9) gates multi-tenant.
151   - [ ] **Data-flywheel harness (M11)** captures?stores?shadow-evals?goldenizes consented
adult jobs; the owned model is on a measured path to the D11 floor.
152   - [ ] ? flip Status DONE, archive to `docs/archives/past_projects/`.
153
154   ## 10. References
155   **ADR lineage:** ADR-005/008/009/010/011/012/013 ? **ADR-014**
(`docs/archives/decisions/DECISIONS.md`). **Strategy:** `PLATFORM_ARCHITECTURE.md`,
`HOSTING_PLAN.md`, `VIDEO_LOCALITY_MODEL.md`, `DESKTOP_APP_PLAN.md`,
`LOCAL_APPLIANCE_ARCHITECTURE.md` (khelsutra). **Archived predecessor:**
`docs/archives/past_projects/DECOUPLED_COMPUTE/` (esp. `05-COST-ATTRIBUTION.md`, `07-COMPUTE-
ABSTRACTION.md`, `06-RISKS-AND-OPEN-QUESTIONS.md`, `DEPLOY_REPORT_GCP_2026-06-20.md`). **Code
anchors `[verified]`:** `backend/pipeline/detectors/base.py:164`, `tracknet_runner.py:281`,
`stitcher/compiler.py:303`, `segmenters/trajectory_hybrid.py:65,237`, `backend/storage/*`,
`backend/worker/__main__.py:88`, `backend/api/{uploads,job_worker}.py`,
`backend/infrastructure/database.py`. **Design provenance:** workflow `wf_56274ff0` (4 code-
mappers ? synthesis; 58 findings).
156
157   ---
158   ### Changelog
159   - 2026-06-21 ? created from the design workflow output; M0 in progress.
160   - 2026-06-21 ? **M0 APPROVED**; **M1 landed**
([#279](https://github.com/Khelsutra/badminton-highlight-indexer/pull/279)):
`deployment.profile` + `compute_target` + preset bundles + precedence/auto-detect-as-suggestion
(D15) on `AppConfig`. Pure/additive, `profile=''` default == today's behaviour (zero core
change); presets touch only existing knobs (no M2 `input_backend` / M3 output-base over-reach).
Suite 1068 green, cov 84.11%.
161   - 2026-06-21 ? **M2 landed** ([#280](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/280)): `input_backend`/`input_root` + URI-aware main API/CLI input via
`resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.
162   - 2026-06-21 ? **M3 landed** ([#282](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/282)): configurable output/scratch base via
`backend/utils/paths.output_base(config)` ? the 3 segmenters' chunk/handoff/temp dirs root under
`output.output_dir` (default `"output"`, byte-identical; default-OFF ? no Launch Report). Fixes
the latent bug where scratch ignored `--output-dir`. Suite 1114 green, cov ~84.9%.
163   - 2026-06-21 ? **M4 landed** ([#283](https://github.com/Khelsutra/badminton-highlight-
indexer/pull/283)): `DeviceContext` + native WASB **CPU-fallback** via a new `device="auto"`
(cuda-if-available-else-cpu); explicit `"cuda"` stays strict (no silent slow-CPU downgrade),
default unchanged. `"auto"` resolves before argv (never reaches `wasb_infer.py`); CUDA probe
cached. Device-selection tested offline; real CPU-inference run owner/GPU-verified. Suite 1126
green, cov 84.96%. **? M1?M4 (the config/input/output/device layer) COMPLETE.**
164   - 2026-06-21 ? **M5+ batch authorized + design picks locked** (owner): the remaining §8
picks resolved ? **D16** (M6 = Cloud Run **Jobs** + bucket-poll; **libx264 encode + NVDEC
decode-only**; **hard-reject `<2GB`** input cap; dir name confirmed) and **D17** (real cloud
verification budget = **$100/?10k cap**, delete-after). Owner gave a blanket go to write+merge
the **default-OFF code** for every remaining gated milestone (M5,M6,M8,M9-auth,M9-consent-
cols,M11-plumbing,M12), one isolated PR each, reserving the gate for the real
run/spend/calibration/counsel. Grounded by gate-map workflow `wf_4849400f`.
165   - 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/288)): per-job **cost ledger** ? `backend/billing.py` (pure `usage ×
pricebook` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on
`jobs` + a versioned `pricebook` table seeded `placeholder-v0` at $0; additive-NULL, idempotent)
+ `record_job_cost`/`get_job_cost`/`get_video_cost` + `GET /api/jobs/{id}/cost` +
`/api/videos/{id}/cost` + `BillingConfig` (**default-OFF**) + a gated `_maybe_record_job_cost`
hook. Adversarial review (`wf_498b24af`) folded a **major silent-GPU-under-bill** (an unpriced
`gpu_type` is now surfaced, not invisible $0) + the **honest no-op** caveat (usage emission is a
follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **? M8
? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a
follow-up; placeholder ships $0 = no user-facing change. v6 is the shared bump (M9-consent cols
extend it).**
166   - 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-
highlight-indexer/pull/287)): `backend/compute/` ComputeBackend registry ? `CloudRunCompute`

dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via `execution_policy.poll_until` + the storage layer (D16); `get_compute_backend` returns a backend ONLY for `compute_target=cloud_run` (else `None` ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced `local_gpu`+native, never re-dispatch); the worker writes a `--done-uri` completion marker on success.
`deploy/cloudrun/{Dockerfile,README}` = the L4 image (CUDA+ffmpeg+fonts+`wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf_87b22fa4`) folded a **blocker** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait test gap + `PolicyTimeout`?clean rc 4. Suite green, ruff+mypy clean. **? M6 ? ? the real build/push/L4 run + DEPLOY_REPORT is M7 (owner, D17 budget).**
167   - 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): `backend/config/startup.py` ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (`_enforce_startup_or_exit` ? exit 1) + the worker (`cmd_infer`/`cmd_train` ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at `profile=""`). **L4 profile-parity test** (`tests/test_profile_parity.py`): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf_34671ac1`). Suite green, ruff+mypy clean. **? M5 ? ? the real-GPU truly-local runlog (L5) is owner-side** (template pre-staged in `runlogs/`).
168

15. user (2026-06-21T13:35:15.231Z)

550-
551-## ADR-011: DECOUPLED_COMPUTE scope = decoupling core (M0?M2 + M3.5); defer serving/billing; DO=training, GCP=serving
552-
553:**Date:** 2026-06-20 · **Status:** RATIFIED (owner, in-session). **Refines ADR-010.** ? **The DO=training / GCP=serving split is SUPERSEDED by [ADR-013](#adr-013-drop-digitalocean-for-compute--gcp-is-the-sole-compute-stack-training--serving)** ? DigitalOcean is dropped for compute; GCP is the sole compute stack. (The scope decision ? DECOUPLED = M0?M2 + M3.5 ? still stands.)
554-
555-### Decision
556-
--
595- staged to Spaces. The old DO CPU droplet `579001481` is drained + destroyed; the exposed Spaces key
596- rotated (ADR-010).
597-
598:## ADR-012: Compute = GPU-native (L4/T4); TPU deferred; Cloud Run + GPU (scale-to-zero) is the supported serving model
599-
600-**Date:** 2026-06-20 · **Status:** RATIFIED (owner, in-session). **Refines ADR-011** (concretizes the "GCP = serving" path).
601-
--
634-- The **deferred M3 (inference serving endpoint + per-video billing) and M4 (scale-to-zero + cost
635- guard)** from ADR-011 become a **new tracked subproject ? "Cloud Run + GPU serving"** ? to be spun
636- up (as a `docs/PROJECT_DOC_TEMPLATE.md`-based doc) **AFTER `docs/DECOUPLED_COMPUTE/` is archived**
637: (owner sequencing). Seed scope below. **? REALIZED in ADR-014 (2026-06-21):** the subproject is
638- `docs/COMPUTE_DECOUPLED_SERVING/`, **renamed** to reflect that **truly-local is a co-equal profile**
639: (not just cloud serving); ADR-014 expands this 7-point seed into the M0?M10 plan.
640-- **First subproject work item = containerize `python -m backend.worker`** for Cloud Run. This is the
641- known hard part: WASB's torch-1.11 conda env makes the image non-trivial (two runtimes ? the
642- indexer venv + the `wasb` conda env). The decoupled storage/compute seams + the native runner

```
--
658-6. NVDEC GPU-decode to kill the extraction bottleneck.
659-7. DPA / consent gate for non-owner uploads (productization, not compute).
660-
661:## ADR-013: Drop DigitalOcean for compute ? GCP is the SOLE compute stack (training +
serving)
662-
663:**Date:** 2026-06-20 · **Status:** RATIFIED (owner, in-session: "drop DO and document it?
record it and bake it for all future work"). **Supersedes the DO=training split in ADR-011;
refines ADR-010 + ADR-012.**
664-
665-### Decision
666-
667-1. **DigitalOcean is DROPPED as a compute provider.** **GCP is the sole compute stack** for
BOTH training and serving. (ADR-011's "DO=training, GCP=serving" split is retired ? GCP does
both. The DECOUPLED scope decision in ADR-011 ? M0?M2 + M3.5 ? still stands.)
668:2. **Standing default for ALL future compute work = GCP** (a GPU VM today via `deploy/gcp/`;
Cloud Run+GPU for serving per ADR-012). Do **not** re-evaluate DO/Paperspace for compute without
a new ADR.
669-3. The `$200` DO credits, if used at all, go **only to non-compute** that DO bills cleanly
(CPU droplets / a site host / Spaces storage / bandwidth) ? **never GPU**.
670-
671-### Rationale (researched 2026-06-20, against DO's own docs)
--
680-- **`deploy\digitalocean\` scripts are KEPT** ? `bootstrap.sh` \ `mount_scratch.sh` \
`gpu_setup.sh` are **provider-agnostic** Linux-GPU setup the GCP runbook reuses; only the DO-
specific *provisioning* (doctl droplet create, Spaces) is dropped. (Optional later cleanup: move
them to `deploy/common/`.)
681-- The shared DO CPU droplet **`claude-do-rally-test`** (another session's testbed) is
**owner-coordinated** cleanup, separate from this decision; the freshly-rotated **DO API token**
can be retired once that droplet is gone.
682-- **Storage stays bucket-abstracted** (`gcs://` primary). The `s3` backend still speaks any
S3-compatible store, so **DO Spaces remains a *possible* storage target** even though DO
*compute* is dropped.
683:- **Capacity reality:** GCP GPU stockouts are common (asia-south1/Mumbai was fully stocked-
out 2026-06-20 ? the M2 run landed an L4 in asia-southeast1/Singapore; weights still land in the
asia-south1 bucket). Sweep zones/regions; scale-to-zero (ADR-012) is part of why Cloud Run+GPU
fits.
684-- **DECOUPLED_COMPUTE** archives on M2 completion with a note that the DO leg was
**dropped** (this ADR), not "delivered on DO".
685-
686:## ADR-014: Compute-decoupled rally serving ? one pure core, profile-selected compute +
storage
687-
688:**Date:** 2026-06-21 · **Status:** ? **RATIFIED (owner, 2026-06-21 ? M0 design sign-off;
build M1+).** **Refines ADR-011 + ADR-012** (it *realizes* their deferred M3/M4; it does **not**
supersede them); **builds on ADR-013** (GCP sole compute stack). Full design:
`docs/COMPUTE_DECOUPLED_SERVING/`.
689-
690-### Evolution / lineage (trace the idea)
691:[Omitted long matching line]
692-
693-### Decision
694-
--
704-- The seams already exist: `_resolve_runner` is the single compute seam, the
`StorageBackend` ABC + 4 backends + the worker fetch/put are built, and the stub detector
(`replay_csv`) gives a **$0 deterministic** core. Realizing a profile system is **composition,
not a rewrite**.
705-- Separating **WHERE** (ComputeBackend, coarse, per-job) from **HOW** (DeviceContext, fine,
per-GPU) from **WHAT-bytes** (StorageBackend) keeps each axis independently testable and avoids
the main trap ? conflating them.
706-- **Metering stitch on Cloud Run** is the only way the owner's real serving cost (GPU + CPU
re-encode + egress) is attributable per job; stitch-on-laptop hides it.
707:- Aligns with ADR-012 (GPU-native) + ADR-013 (GCP sole stack) + the 2026-06-20 deploy
```

```
report; the truly-local profile preserves the ADR-009 privacy/offline self-host story.
708-
709-### Consequences
710-
```

16. assistant (2026-06-21T13:35:18.845Z)

I have the full picture of the config system. Let me verify how the profile/compute_target are actually consumed at runtime in the worker and compute backend, to ground the "single build runs both" claim.

17. user (2026-06-21T13:35:21.331Z)

```
1       """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3       COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4       ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5       otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6       identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7       changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8       """
9
10      from __future__ import annotations
11
12      import os
13      from typing import Any, Optional
14
15      from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16      from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18      __all__ = [
19          "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20      ]
21
22
23      def _storage_dict(config: Any) -> dict:
24          storage = getattr(config, "storage", None)
25          if storage is None:
26              return {}
27          if hasattr(storage, "model_dump"):
28              return storage.model_dump()  # type: ignore[no-any-return]
29          return dict(storage) if isinstance(storage, dict) else {}
30
31
32      def get_compute_backend(
33          config: Any,
34          *,
35          run_client: Optional[Any] = None,
36          policy: ExecutionPolicy = DEFAULT_POLICY,
37          token: Optional[str] = None,
38      ) -> Optional[ComputeBackend]:
39          """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40          in-process (the default-OFF path).
41
42          Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43          Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44          / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
```

```
45          pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46          """
47          target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48          if target != "cloud_run":
49              return None  # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51          # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52          from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54          job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55          region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56          project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57          client = run_client if run_client is not None else GoogleCloudRunJobClient()
58          return CloudRunCompute(
59              run_client=client, job_name=job_name, region=region, project=project,
60              storage_config=_storage_dict(config), policy=policy, token=token,
61          )
62
```

18. user (2026-06-21T13:35:21.472Z)

```
backend\compute\__init__.py
backend\compute\cloud_run.py
backend\compute\base.py
```

19. user (2026-06-21T13:35:21.962Z)

```
backend\worker\__main__.py:4:        [--segmenter wasb_hybrid] [--config config.json] [--set k=v
...] [--scratch DIR]
backend\worker\__main__.py:10:``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-
free** run (no Gemini key) ?
backend\worker\__main__.py:28:from backend.compute import ComputeJobSpec, get_compute_backend
backend\worker\__main__.py:31:    enforce_startup_checks,
backend\worker\__main__.py:46:    """Coerce a ``--set`` string value to bool/int/float where it
round-trips, else str."""
backend\worker\__main__.py:63:            raise ValueError(f"--set expects k=v, got {kv!r}")
backend\worker\__main__.py:110:        enforce_startup_checks(config, cuda_available=cuda,
logger=logger)
backend\worker\__main__.py:113:        return False  # already logged at ERROR by
enforce_startup_checks
backend\worker\__main__.py:119:    container runs this SAME cmd_infer INLINE (compute_target
forced local_gpu) ? never re-dispatch.
backend\worker\__main__.py:162:    # M6: a cloud compute_target DISPATCHES to a Cloud Run Job
instead of running in-process.
backend\worker\__main__.py:163:    # DEFAULT-OFF: get_compute_backend returns None for
""/local_gpu/cpu_mock ? fall through to the
backend\worker\__main__.py:165:    # dispatcher forces compute_target=local_gpu), so it never
recursively re-dispatches.
backend\worker\__main__.py:166:    compute = get_compute_backend(config)
backend\worker\__main__.py:467:    pi.add_argument("--set", action="append", default=[],
metavar="k=v",
backend\worker\__main__.py:468:                    help="config override, e.g. --set
indexing.skip_ai_handoff=true")
backend\worker\__main__.py:488:    pt.add_argument("--set", action="append", default=[],
metavar="k=v")
```

20. assistant (2026-06-21T13:35:27.483Z)

Let me read the key parts of the worker `__main__.py` to ground how `--set`, the env override, and the compute dispatch wire together at runtime.

21. user (2026-06-21T13:35:28.499Z)

```
    from backend.utils.hashing import compute_video_id

    logger = logging.getLogger("backend.worker")


    def _coerce(v: str) -> Any:
        """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
        low = v.lower()
        if low in ("true", "false"):
            return low == "true"
        for cast in (int, float):
            try:
                return cast(v)
            except ValueError:
                pass
        return v


    def _apply_overrides(config: Any, sets: List[str]) -> None:
        """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
        for kv in sets or []:
            key, sep, val = kv.partition("=")
            if not sep:
                raise ValueError(f"--set expects k=v, got {kv!r}")
            parts = key.split(".")
            obj = config
            for p in parts[:-1]:
                obj = getattr(obj, p)
            setattr(obj, parts[-1], _coerce(val))


    def _write_intervals_csv(segments: List[dict], path: str) -> None:
        """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
        artifact (same schema as the rally-annotator labels)."""
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["start", "end", "shot_count", "ending_reason"])
            for s in segments:
                w.writerow([
                    f'{float(s.get("start_time", 0.0)):.3f}',
                    f'{float(s.get("end_time", 0.0)):.3f}',
                    s.get("shot_count", ""),
                    s.get("ending_reason", s.get("shot_count_confidence", "")),
                ])


    def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
        with open(path, "w") as f:
            json.dump({
                "video_id": video_id,
                "status": status,
                "segment_count": len(segments),
                "segments": [{"start": float(s.get("start_time", 0.0)),
                              "end": float(s.get("end_time", 0.0))} for s in segments],
            }, f, indent=2)


    def _run_startup_checks(config: Any) -> bool:
        """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
        best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
```

```
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None  # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False  # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
120
121         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
122         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
123         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
124         from backend.infrastructure.execution_policy import PolicyTimeout
125
126         spec = ComputeJobSpec(
127             video_uri=args.video_uri, out_uri=args.out_uri,
128             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
129         )
130         try:
131             result = compute.run_infer(spec)
132         except PolicyTimeout as e:
133             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
134             return 4
135         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
136                     result.video_id, result.returncode, result.outputs_uri)
137         return result.returncode
138
139
140     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
141                            status: str, storage_cfg: dict, scratch: str) -> None:
142         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
143         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
144         try:
145             marker = {"video_id": video_id, "returncode": returncode,
146                       "segment_count": segment_count, "status": status}
147             local = os.path.join(scratch, "_done.json")
148             with open(local, "w", encoding="utf-8") as f:
```

```
149                 json.dump(marker, f)
150             storage.put(local, done_uri, config=storage_cfg)
151             logger.info("[worker] wrote completion marker -> %s", done_uri)
152         except Exception as e:  # never fail a good run because the marker push hiccuped
153             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
154
155
156     def cmd_infer(args: argparse.Namespace) -> int:
157         config = load_config(args.config) if args.config else load_config()
158         _apply_overrides(config, args.set)
159         if not _run_startup_checks(config):
160             return 2
161
162         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
163         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
164         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
165         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
166         compute = get_compute_backend(config)
167         if compute is not None:
168             return _dispatch_remote(compute, args)
169
```

22. assistant (2026-06-21T13:35:35.388Z)

I now have a complete, code-grounded picture. Let me also quickly confirm how the API server entrypoint wires startup checks and that there's no `--set` precedence over env at the server (to be honest about what the packaged app exposes).

23. user (2026-06-21T13:35:35.699Z)

```
33:    load_config as load_app_config,
36:    enforce_startup_checks,
101:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
486:    cfg = load_config()  # legacy wrapper still works, returns dict for now
552:def _enforce_startup_or_exit(config) -> None:
559:        enforce_startup_checks(config, logger=logger)
639:    _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile
```

24. user (2026-06-21T13:35:35.801Z)

```
backend\pipeline\detectors\native_wasb_runner.py:17:    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` /
``WASB_PYTHON`` / ``WASB_SCRATCH``), then
backend\pipeline\detectors\native_wasb_runner.py:83:        env = os.environ.get
backend\pipeline\detectors\native_wasb_runner.py:89:            python_bin=env("WASB_PYTHON") or
w.get("python_bin", "python"),
backend\pipeline\detectors\native_wasb_runner.py:92:            device=env("WASB_DEVICE") or
w.get("device", "cuda"),
backend\pipeline\detectors\native_wasb_runner.py:175:            return False, f"python binary
not found: {self.cfg.python_bin!r} (set WASB_PYTHON)."
backend\pipeline\detectors\native_wasb_runner.py:239:            logger.error("python binary not
found: %r (set WASB_PYTHON).", self.cfg.python_bin)
backend\pipeline\segmenters\gemini.py:90:        api_key = os.environ.get(api_key_env_var)
backend\pipeline\detectors\stub_runner.py:20:``indexing.detector_impl="stub"`` or the
``RALLY_DETECTOR_IMPL=stub`` env var; the real
backend\pipeline\segmenters\trajectory_hybrid.py:69:        ``RALLY_DETECTOR_IMPL`` env var
(takes precedence) or ``indexing.detector_impl``:
backend\pipeline\segmenters\trajectory_hybrid.py:77:        impl =
(os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or "auto").lower()
backend\pipeline\segmenters\trajectory_hybrid.py:169:            api_key =
os.environ.get(api_key_env_var)
```

```
backend\pipeline\segmenters\trajectory_hybrid.py:191:        # _resolve_runner honours the CPU-
stub override (RALLY_DETECTOR_IMPL /
backend\pipeline\segmenters\yolo_hybrid.py:80:        api_key = os.environ.get(api_key_env_var)
```

25. assistant (2026-06-21T13:35:40.435Z)

I have everything I need. Let me confirm the `git pull` failed state is just a local multi-branch fetch config issue (not blocking my read-only analysis), then synthesize.

The `git pull --ff-only` failed with "Cannot fast-forward to multiple branches" ? a local fetch-refspec quirk, not a divergence on my branch; my analysis is read-only against the checked-out tree, so this doesn't affect correctness. I have all the grounding I need.

26. user (2026-06-21T13:36:54.034Z)

Output does not match required schema: root: must have required property 'key_facts'

27. assistant (2026-06-21T13:37:04.452Z)

The schema validation requires the top-level fields directly. Let me re-call with the correct shape ? the previous call had the right keys, so the error likely indicates a structural issue with how arrays/objects were nested. Let me retry exactly with all required properties at root.

28. user (2026-06-21T13:37:51.478Z)

Output does not match required schema: root: must have required property 'packaging_implications'

29. assistant (2026-06-21T13:37:55.011Z)

The error is now flagging `packaging_implications` (the prior one flagged `key_facts`). The issue: I used `<parameter name=` (without the `antml:` prefix) for `summary`, `packaging_implications`, `risks`, and `open_questions`, so those keys weren't parsed. Let me redo with consistent `antml:parameter` tags throughout.

30. user (2026-06-21T13:38:40.217Z)

Structured output provided successfully

31. assistant (2026-06-21T13:38:42.341Z)

Structured output delivered successfully.